JIE QI

Service Engineer

qjwd778@gmail.com (+86)18961994578 Suzhou

Summary

Full-stack and AI application engineer with an M.Sc . in Computer Science from the University of Liverpool, combining hands-on backend and AI delivery experience with a genuine interest in client-facing, business-oriented work. Comfortable communicating across cultures — independently handled on-site reception and technical exchanges with visiting European and African clients, uncovering real needs through conversation and building lasting relationships. Seeking to grow toward a pre-sales / customer-facing role, translating customer pain points into clear technical solutions and driving them to delivery.

Core Strengths

- **Cross-cultural communication & client relations** — Overseas study background and strong English; independently led on-site reception and technical exchanges with international clients (Europe, Africa) and maintained long-term relationships, building trust across diverse cultural backgrounds.
- **Needs discovery & value articulation** — Skilled at digging into customer needs, identifying true pain points, and highlighting product/solution strengths to strengthen customer confidence.
- **Empathetic, situational communication** — Attuned to a counterpart's state and priorities; adapts tone and message to the person and context for more effective conversations.
- **Research & synthesized judgment** — Habitually compares multiple sources, distills key signals from reviews and feedback, and weighs trade-offs to reach sound decisions.
- **Fast learning & problem-solving** — Quickly builds a working methodology and picks up new skills under time pressure; approaches problems from multiple angles to find a way through.

Language

English
IELTS 6.5 (Speaking 7.0)

Chinese

Technical Skills

- **AI-Assisted Development** — Use Claude Code / Cursor daily for requirement breakdown, coding, refactoring, and bug fixing, with real experience shipping AI-generated code to production; review, test, validate, and correct AI output, taking ownership of its correctness, performance, and security.
- **SDD / TDD Practice** — Established and applied a "spec-first, test-first" workflow to constrain AI coding: acceptance criteria up front, followed by a four-step loop of mandatory real testing, progress logging, and commit — keeping AI code controllable, traceable, and free of unfounded assumptions.
- **AI Engineering Standards** — Distilled reusable prompts, spec templates, project rules, AI Skills, and MCP tools into a team-reusable AI development standard.
- **Java Backend** — Proficient in Spring Boot at a production lead-developer level; strong with MySQL and Redis; RESTful API design, Controller-Service-DAO layering, validation, authentication, and data persistence.
- **CS Fundamentals** — Solid grasp of networking, concurrency, transactions, caching, and common distributed-system design; proven ability to diagnose race conditions, deadlocks, and stability/performance issues.
- **AI Applications / Agent Architecture** — RAG (retrieval-augmented generation), vector databases, embeddings, MCP tool calling, multi-turn dialogue, and context management; delivered a hybrid RAG + Agent architecture.

- **Engineering & Deployment** — Docker, Docker Compose, Kubernetes, Linux, and Git; able to deliver and ship full-stack projects independently with AI assistance.

Professional Experience

Suzhou Aiyisheng Automation Technology Co., Ltd., 2025 – Present
Robotics Application Engineer Suzhou

- **Product reception and technical exchanges for visiting European and African clients** — presented products and technical solutions, answered on-site questions, and built long-term relationships afterward, gaining hands-on experience in communicating with, presenting to, and maintaining international clients.
- **Built the AI-robot business backend** on Spring Boot + MySQL + Redis, independently owning the API, database, and caching design for device management, agent configuration, and device binding — from design through development to release.
- Applied **Claude Code / Cursor** in daily development for requirement breakdown, coding, refactoring, and bug fixing, reviewing, testing, and correcting AI-generated code before shipping.
- Built a **RAG platform**, applying an **SDD/TDD** workflow to constrain AI coding in a real project, and distilled reusable prompts, spec templates, project rules, and MCP tools for the team.
- **Diagnosed and resolved a range of backend issues** — device-registration infinite loop, concurrency race conditions and flow-control deadlocks, and cross-service authentication incompatibility — demonstrating strength in concurrency, caching, and stability troubleshooting.

Changzhou Senpu Technology Co., Ltd., *Full-Stack Engineer (Internship)* 2022 – 2024
Changzhou Jiangsu

- **Developed mobile-side Flutter components and page features**, delivering reusable component encapsulation and interaction-logic optimization.
- Built and maintained **Flask API backend** interfaces, supporting mobile business features and backend data processing.
- Handled **PostgreSQL** backup, migration, and routine maintenance, ensuring data safety and stable environment switching.
- Supported iteration through **feature integration**, **troubleshooting**, and deployment, improving system maintainability and delivery efficiency.

Education

M.Sc., Computer Science, *University of Liverpool* 2024 – 2025
Research focus: Deep Reinforcement Learning algorithms UK

B.Eng., Software Engineering, *Changzhou Institute of Technology* 2020 – 2024
Outstanding Graduation Thesis · Outstanding Graduate · President, Programmers' Club China

Projects

Enterprise RAG Knowledge-Base Q&A Platform, *Python, FastAPI, LangChain, PostgreSQL, Qdrant, Redis, MinIO, MCP, Vue 3, Docker/K8s · Built with Claude Code + SDD/TDD* 01/2026 – Present

- Delivered the full **front-to-back stack**, applying a spec-first, test-first workflow, and released it for internal pilot use by real users (finance/tax staff).
- Designed a hybrid **RAG + Agent** architecture: document parsing → chunking → BGE-M3 embedding → Qdrant retrieval + rerank + recency filtering → streaming LLM generation with citations; built an MCP / function-calling tool to connect the knowledge base to a voice agent, enabling tool calls and context management.
- Designed the business system: multi-tenant API with **MySQL/PostgreSQL** and Redis caching, including JWT / API-Key dual authentication, RBAC, row-level isolation, quota-based rate limiting, and end-to-end auditing — handling stability and concurrency concerns.
- Owned **AI code quality**: all AI-generated code went live only after code review and real testing, intercepting and correcting multiple AI misjudgments.

Robot Management & Voice Service, *Spring Boot, MyBatis-Plus, MySQL, Redis, Python asyncio, WebSocket, MQTT, OTA, ASR/LLM/TTS* 01/2026 – Present

- Developed the **backend** for device management, agent configuration, and voice-agent communication, engineering core Agent capabilities: multi-turn dialogue, intent recognition, function-call tool use, and context/memory management.
- **Fixed a device-registration infinite-loop risk**: the legacy logic did not regenerate the Redis key on verification-code collision; reordered generation and lookup to regenerate on every collision, improving registration-endpoint stability.
- **Resolved authentication incompatibility** between the Java **OTA endpoint** and the Python **WebSocket service** by aligning the AuthManager signing rules — implementing clientId|deviceId|timestamp HMAC-SHA256 token generation on the Java side and delivering it via OTA for consistent cross-service authentication.
- **Optimized device binding/activation**: cached activation code, MAC, hardware model, and firmware version in Redis, persisting user–device–agent relationships and clearing caches on success to prevent dirty data.
- **Prevented resource waste from unauthorized devices**: refined the Python ConnectionHandler binding-state check to drop audio before binding is confirmed and prompt only at intervals, keeping unauthorized devices out of the **ASR/LLM/TTS** pipeline and cutting wasted compute and token cost.
- Fixed **voice-queue** race conditions and TTS flow-control deadlocks via binding-state events, send-queue state, and sentence_id reset — resolving incomplete playback and deadlocks after interruption-then-rewake, improving observability and stability.
- Refactored **ASR** audio preprocessing, extracting duplicated Opus decoding, PCM merging, and temp-file handling across providers into a shared base class (AudioArtifacts), reducing duplication and maintenance cost across multiple models.